Numérique Durable (TI616)

MINI-PROJET HACKATHON : « LE SITE LE PLUS GREEN »

Source: https://welovedevs.com/fr/app/tests/greenit-and-greenweb

Participants

Grecie Ornella TETE NDOM

Charles Kpéra YERIMA

Hugo WILT

Sous l'encadrement de:

Yvan GUIFO FODJO

ING1- 2028

Table des matières

Section 1. Présentation du projet 3

1) Proposition de valeur 3
2) Périmètre MVP 3
3) Fonctionnalités incluses 3
4) Utilisateurs cibles 4
5) Contraintes Green IT 4

Section 2. Architecture et conception 5

1) Diagramme de cas d'utilisation 5
2) Diagramme de classes 5
3) Diagramme de séquence 6
4) Schéma de la base de données 6
5) Architecture générale (front / back / BDD / déploiement) 7

Section 3. Choix technologiques justifiés 9

Section 4. Implémentation 10

Section 5. Analyse de l'empreinte carbone 12

Section 6. Tests et validation 16

Section 7. Organisation de l'équipe et collaboration 22

Section 8. Discussion et conclusion 23

1. Défis rencontrés 23
2. Compromis entre fonctionnalités et sobriété 23
3. Pistes d'amélioration 24

Section 9. Annexes 25

Section 1. Présentation du projet

1) Proposition de valeur

GreenQuiz est une plateforme éducative de quiz interactifs dédiée à l'éco-conception web. Elle permet à ses utilisateurs (étudiants et professionnels) d'apprendre les bonnes pratiques du Green IT de manière ludique, en répondant à des quiz thématiques portant sur l'optimisation des images, la sobriété CSS, les bases de données légères ou encore les bonnes pratiques de développement responsable tout en limitant l'utilisation de ressources.

2) Périmètre MVP

Pour le projet, il suffira d'avoir un quiz interactif et complet en utilisant de l'HTML, CSS et JavaScript, ainsi qu'une base de données de questions et un classement pour donner envie aux utilisateurs de plus apprendre. Le site n'a pas besoin de complexité, un système de points, de bonne ou mauvaise réponse, de feedback sont les éléments clés pour une première version. L'utilisateur peut ainsi créer un compte, un nouveau quiz, rechercher des quiz ou encore modifier/supprimer son propre quiz pour pousser les utilisateurs à s'engager. Le MVP doit permettre aux utilisateurs de s'inscrire, répondre à des quiz, et voir leurs scores, avec un contenu géré par l'administrateur.

3) Fonctionnalités incluses

Les fonctionnalités principales en vue de la première version du site sont :

- Inscription utilisateur
- Connexion et déconnexion
- Gestion du profil (voir/modifier/supprimer son compte)
- Gestion des utilisateurs (admin)
- Création de quiz Green IT
- Modification et suppression de quiz
- Consultation de la liste des quiz
- Passage des quiz et calcul du score
- Historique des sessions de quiz

4) Utilisateurs cibles

Le public principal est avant tout les étudiants, les professionnels du milieu informatique, mais aussi les curieuses et curieux qui souhaitent en apprendre plus sur le Green IT et comment implémenter cette démarche dans la création d'un site web.

5) Contraintes Green IT

L'équipe a défini cinq contraintes dès la phase de conception.

Le poids de chaque page doit rester inférieur à 200 Ko afin de réduire la consommation réseau et énergétique.

Le nombre de requêtes HTTP par page est limité à 15, car chaque requête consomme de l'énergie côté serveur et client.

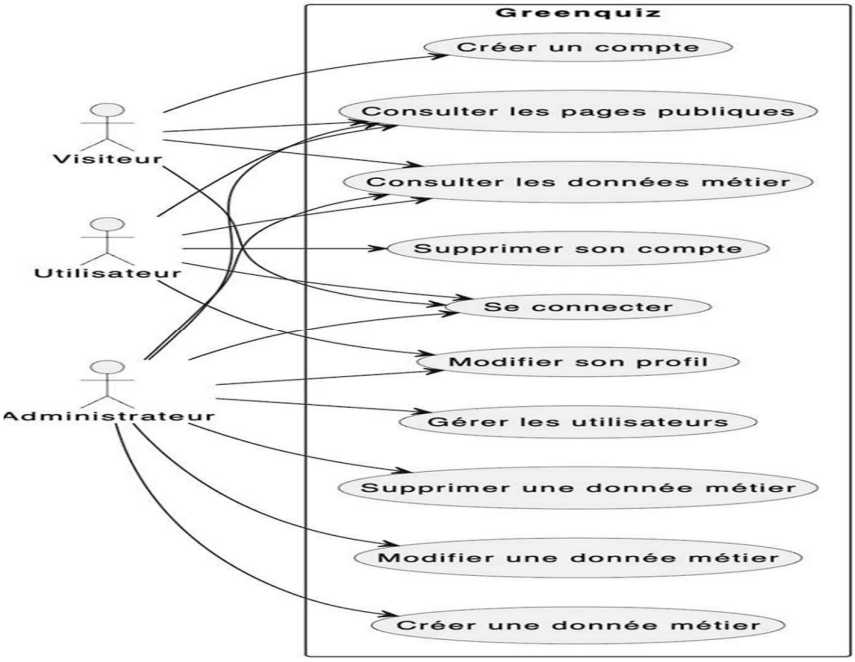
Le score Lighthouse Performance doit dépasser 80/100, garantissant à la fois performance et accessibilité.

Le temps de chargement cible est inférieur à 1 seconde.

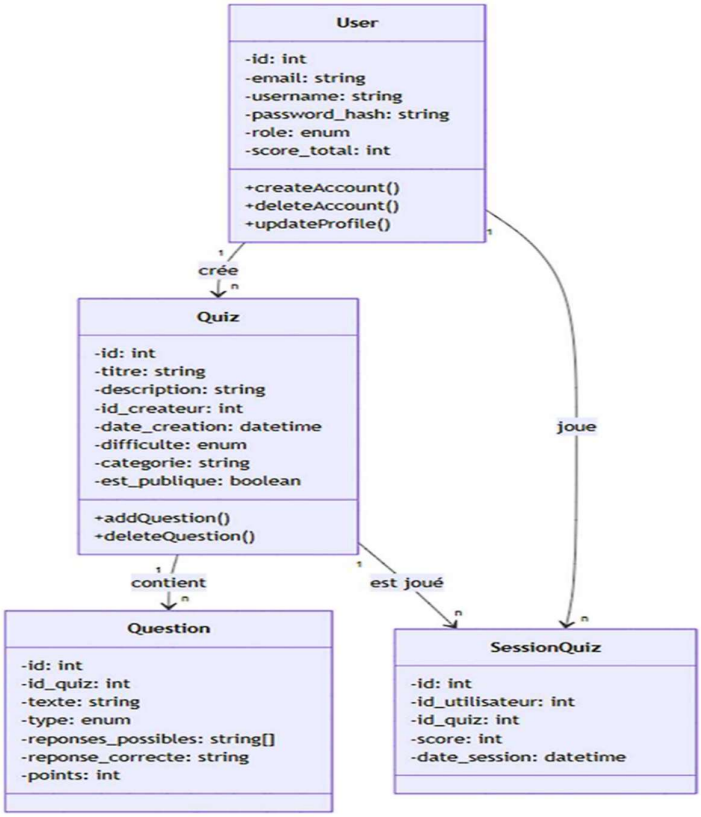
Enfin, seules les données strictement nécessaires sont stockées : pas d'adresse IP, pas de cookies de tracking.

Section 2. Architecture et conception

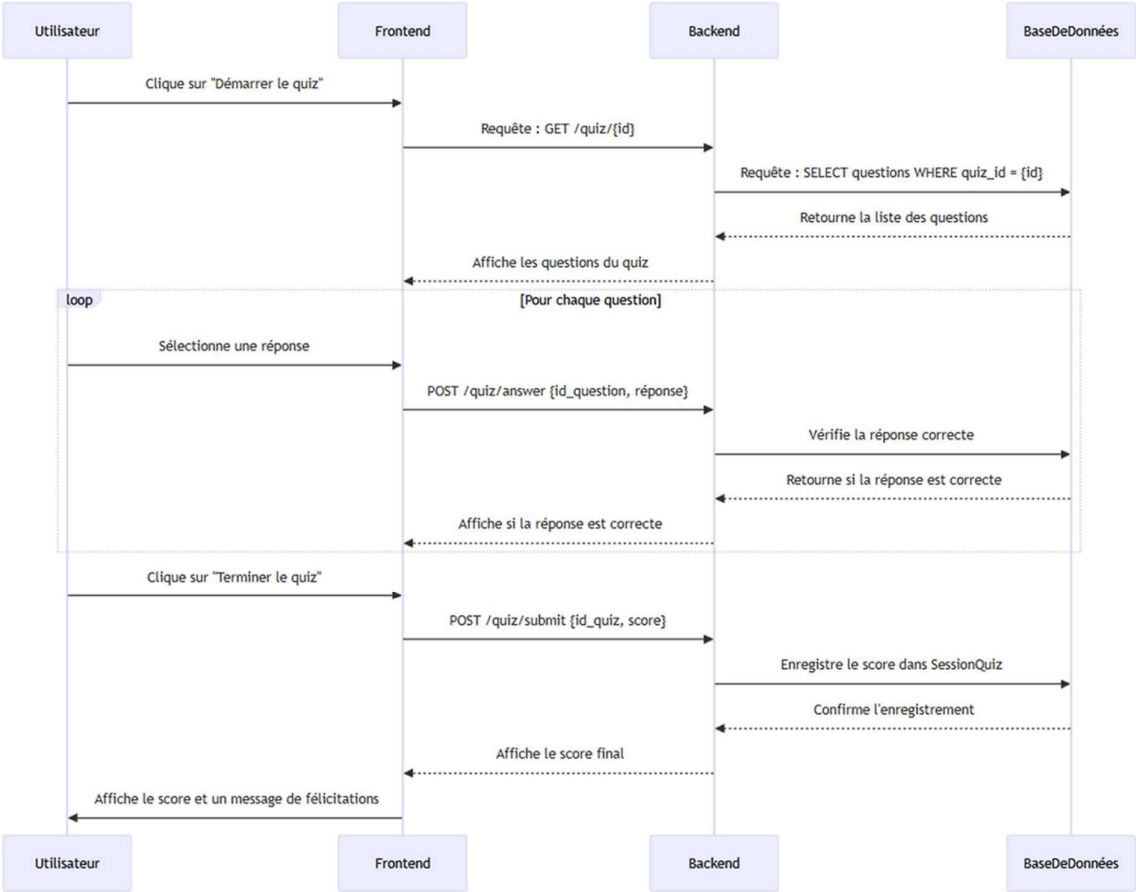
1) Diagramme de cas d'utilisation



2) Diagramme de classes



3) Diagramme de séquence



4) Schéma de la base de données

Tables

- users
    - id INTEGER, PK
    - email TEXT, NOT NULL, UNIQUE
    - username TEXT, NOT NULL
    - password_hash TEXT, NOT NULL
    - created_at TEXT, NOT NULL, default CURRENT_TIMESTAMP
    - is_admin INTEGER, NOT NULL, default 0

- quizzes
    - id INTEGER, PK
    - titre TEXT, NOT NULL
    - description TEXT, NOT NULL
    - id_createur INTEGER, NOT NULL, FK -> users.id (ON DELETE CASCADE)
    - date_creation TEXT, NOT NULL, default CURRENT_TIMESTAMP
    - difficulte TEXT, NOT NULL, CHECK in ('facile','moyen','difficile')
    - categorie TEXT, NOT NULL
    - est_publique INTEGER, NOT NULL, default 1
    - nombre_questions INTEGER, NOT NULL, default 0
- questions
    - id INTEGER, PK
    - id_quiz INTEGER, NOT NULL, FK -> quizzes.id (ON DELETE CASCADE)
    - texte TEXT, NOT NULL
    - type TEXT, NOT NULL, CHECK in ('QCM','vrai/faux','reponse_courte')
    - reponses_possibles TEXT, NULL (JSON string)
    - reponse_correcte TEXT, NOT NULL
    - points INTEGER, NOT NULL, default 1
- sessions_quiz
    - id INTEGER, PK
    - id_utilisateur INTEGER, NULL, FK -> users.id (ON DELETE SET NULL)
    - id_quiz INTEGER, NOT NULL, FK -> quizzes.id (ON DELETE CASCADE)
    - score INTEGER, NOT NULL
    - created_at TEXT, NOT NULL, default CURRENT_TIMESTAMP

5) Architecture générale (front / back / BDD / déploiement)

- **Front-end**
    - Rendu serveur avec Flask + templates Jinja (templates/*.html), base commune via base.html
    - UI en HTML/CSS/JS natif (static/assets/style.css, static/assets/app.js), sans framework lourd
    - Le CSS principal est injecté inline depuis app.py pour réduire les requêtes critiques au chargement

- **Back-end**
  - Application monolithique dans app.py (routes, logique métier, auth, admin, quiz)
  - Gestion session utilisateur via Flask session (session["uid"]), mots de passe hashés avec werkzeug.security
  - Contrôles d'accès avec rôle admin (is_admin) + admin_required.
  - Domaine principal : inscription/connexion, CRUD utilisateurs, CRUD quiz/questions, passage de quiz et scoring

- **Base de données**
  - SQLite local (database/greenquiz.sqlite) avec 4 tables cœur :
    - users
    - quizzes
    - questions
    - sessions_quiz
  - Relations clés :
    - 1 utilisateur crée plusieurs quiz (quizzes.id_createur -> users.id)
    - 1 quiz contient plusieurs questions (questions.id_quiz -> quizzes.id)
    - historique des scores via sessions_quiz
  - Index ajoutés pour les recherches/listes (quiz publics, catégorie, jointures fréquentes).

- **Déploiement**
  - Hébergement sur **Render** (web service).
  - Point important corrigé : serveur Flask écoute sur 0.0.0.0 et utilise PORT (env Render), pas 127.0.0.1.
  - Lancement actuel via python3 app.py (serveur Flask intégré).
    - Fonctionne pour un petit projet/POC.
    - Pour un vrai usage prod, recommandé ensuite : gunicorn (WSGI) + éventuellement disque persistant si SQLite doit être conservée entre redéploiements.

Section 3. Choix technologiques justifiés

Technologies et impacts

Composant	Technologie choisie	Justification Green IT	Compromis
Back-end	Python + Flask	Framework minimal: peu de surcouche, peu de dépendances, temps de dev court donc dette technique réduite	Moins de fonctions natives qu'un framework full-stack
Base de données	SQLite	Base légère sans serveur dédié: consommation infra et maintenance faible pour un petit trafic	Scalabilité limitée en écriture concurrente
Front-end	Templates Jinja + HTML/CSS/JS natifs	Pas de bundle JS lourd ni runtime front complexe: moins de transfert réseau et exécution CPU cote client	Moins de composants prêts à l'emploi
Style	CSS unique (inline au rendu)	Suppression d'une requête critique au premier affichage, amélioration LCP/FCP sur hébergement a latence modérée	Cache navigateur plus limite pour ce CSS
Authentification	Sessions Flask + hash mot de passe Werkzeug	Approche simple, robuste, sans service externe supplémentaire	Fonctionnalités IAM avancées non incluses
Hébergement	Render (web service)	Mutualisation de l'infra et déploiement simple, utile pour un projet sobre et pédagogique	Performance variable et cold starts en free tier
Mesure performance	Lighthouse + optimisation chemin critique	Pilotage par indicateurs concrets (LCP/FCP), réduction des ressources bloquantes	Mesure ponctuelle à compléter par suivi régulier

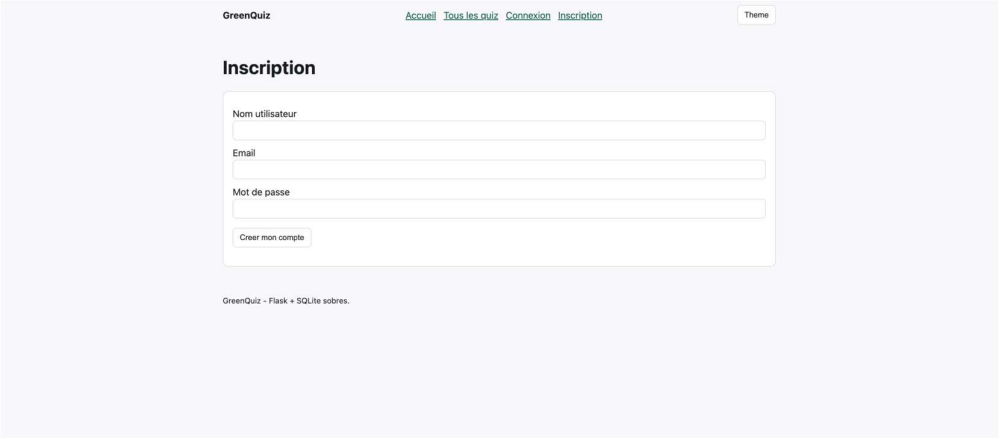
Section 4. Implémentation

Description des fonctionnalités développées. Extraits de code commentés pour les parties les plus significatives (CRUD, requêtes BDD, optimisations).

<u>Fonctionnalités développées :</u>

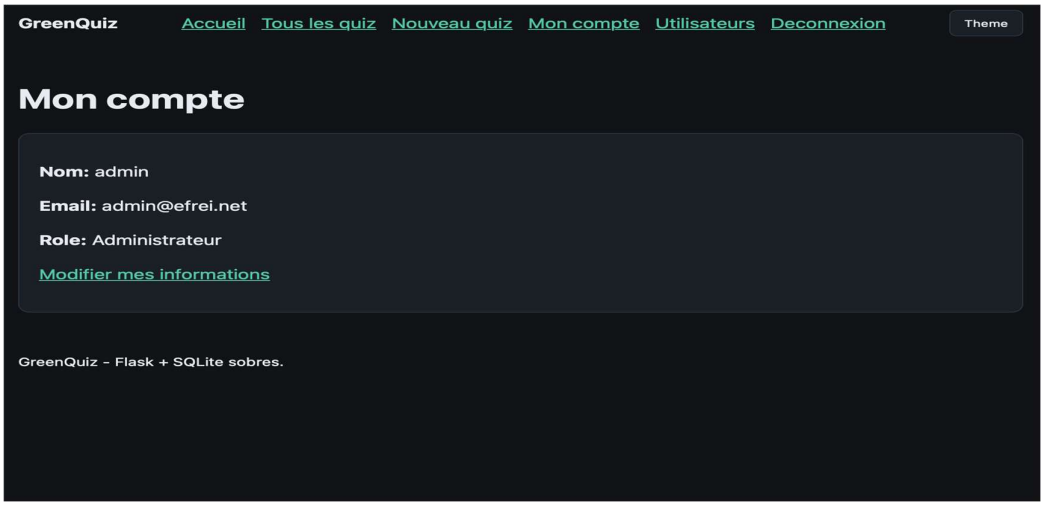
Inscription utilisateur : création d'un nouveau compte pour qu'une personne puisse s'enregistrer sur le site à l'aide de son Nom, son courriel et un mot de passe.

Elle implique généralement la validation des champs le hachage de mots de passe puis l'insertion des données dans la table des comptes

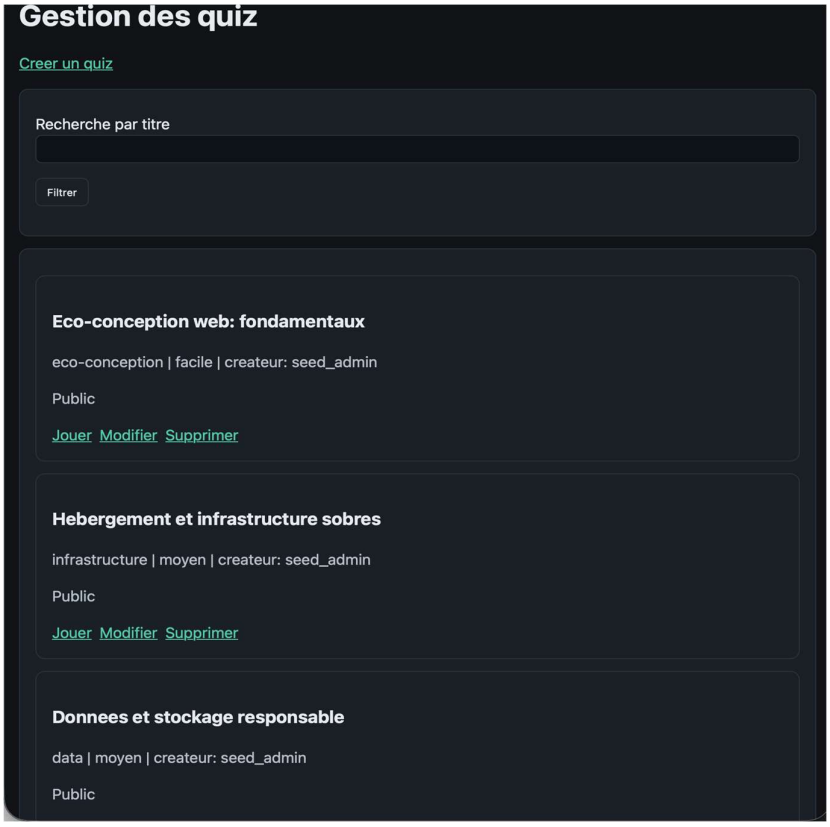


Connexion / déconnexion : la connexion permet à l'utilisateur d'entrer son compte et ses identifiant et la déconnexion permet d'en sortir. La connexion vérifie les identifiants, crée une session ou un token d'accès tandis que la déconnexion invalide cette session ou ce token pour sécuriser l'accès.

Page compte (profil connecté) : c'est la page personnelle affichée lorsque l'utilisateur est connecté accessible uniquement après l'authentification. Il peut voir ses informations et les modifier.



Mini quiz green IT en front (JS local) : composant interactif côté client développé en JavaScript exécuté localement dans le navigateur. Il permet de gérer la logique du quiz sans requêtes serveur, avec stockage éventuel des réponses en mémoire du navigateur ou via localStorage, dans une logique orientée front-end.



Section 5. Analyse de l'empreinte carbone

Page d'accueil :

Indicateur	Avant	Après	Gain
Poids de la page d'accueil	1.9 Ko	2.0 Ko	- 0 %
Nombre de requêtes HTTP	4	2	
Score EcoIndex	95/100	92/100	-3 Pts
Note EcoIndex	A+ (A-G)	A+ (A-G)	
CO2 / visite	< 0.01 g	< 0.01 g	0 %
Score Lighthouse Perf	99/100	100/100	+ 0 Pts
FCP	0.8 s	0.8 s	- 0 %
LCP	0.9 s	0.8 s	- 11 %

Page quizzes :

Indicateur	Avant	Après	Gain
Poids de la page d'accueil	1.4 Ko	1.4 Ko	- 0 %
Nombre de requêtes HTTP	3	2	
Score EcoIndex	95/100	92/100	-3 Pts
Note EcoIndex	A+ (A-G)	A+ (A-G)	
CO2 / visite	< 0.01 g	< 0.01 g	0 %
Score Lighthouse Perf	100/100	100/100	+ 0 Pts
FCP	0.8 s	0.8 s	- 0 %
LCP	0.9 s	0.8 s	- 11 %

Page d'inscription :

Indicateur	Avant	Après	Gain
Poids de la page d'accueil	0.4 Ko	0.4 Ko	- 0 %
Nombre de requêtes HTTP	3	2	
Score EcoIndex	95/100	96/100	+ 1 Pts
Note EcoIndex	A+ (A-G)	A+ (A-G)	
CO2 / visite	< 0.01 g	< 0.01 g	0 %
Score Lighthouse Perf	100/100	100/100	+ 0 Pts
FCP	0.8 s	0.8 s	- 0 %
LCP	0.8 s	0.8 s	- 0 %

Problèmes remarqués :

INSIGHTS

▲ Arborescence du réseau

Évitez les chaînes de requêtes critiques en réduisant la longueur des chaînes ou la taille de téléchargement des ressources, ou en reportant le téléchargement de ressources inutiles, afin d'améliorer le chargement des pages. LCP Non noté

Latence de chemin d'accès critique maximale : **836 ms**

Navigation initiale

https://greenproject-1f1e.onrender.com - **642 ms,** 1,09 KiB

...assets/style.css (greenproject-1f1e.onrender.com) - **836 ms,** 1,15 KiB

Origines préconnectées

Les hints de préconnexion aident le navigateur à établir une connexion plus tôt dans le chargement de page, ce qui permet de gagner du temps lors de la première requête pour cette origine. Voici les origines auxquelles la page s'est préconnectée.

aucune origine n'a été préconnectée

Candidats pour la préconnexion

Ajoutez des hints de préconnexion à vos origines les plus importantes, mais essayez de ne pas en utiliser plus de quatre.

Aucune autre origine n'est un bon candidat pour la préconnexion

▲ Requêtes de blocage de l'affichage — Économies estimées : 300 ms

Les requêtes bloquent le rendu initial de la page, ce qui peut causer un retard LCP. Le report ou l'intégration de ces requêtes réseau peut les écarter du chemin critique. LCP FCP Non noté

URL	Taille de transfert	Durée
onrender.com Propriétaire	1,1 KiB	150 ms
...assets/style.css (greenproject-1f1e.onrender.com)	1,1 KiB	150 ms

Modifications réalisées :

Au départ, la page était chargée en deux étapes critiques : le navigateur récupérait le HTML, puis devait faire une seconde requête pour télécharger style.css. Comme le CSS est une ressource de rendu, cette requête bloquait l'affichage initial. C'est ce que Lighthouse signalait avec les "requêtes de blocage de l'affichage" et l'"arborescence du réseau" (chaîne critique HTML puis CSS).

Pour corriger ça, nous avons supprimé cette requête CSS du chemin critique. Concrètement, au lieu de charger la feuille de style via une balise <link>, nous injectons le contenu CSS directement dans le HTML de base (base.html), grâce à une variable fournie par app.py (site_css via un context processor). Le style reste centralisé dans un seul fichier source, mais il est livré dans la première réponse HTML.

Résultat : le navigateur n'attend plus une requête supplémentaire pour commencer le rendu, la chaîne réseau critique est plus courte, et le chargement visuel initial s'améliore (notamment sur FCP/LCP).

En modifiant le code, on remarque que le score EcoIndex diminue sur certaines pages :

Ce qu'on a optimisé avec le CSS inline, c'est surtout la vitesse de rendu initial (moins de requêtes critiques, meilleure chaîne réseau). Mais EcoIndex ne mesure pas la même chose que Lighthouse. EcoIndex pénalise davantage la complexité globale de la page (taille du DOM, volume total transféré, nombre de requêtes), pas seulement le chemin critique.

En mettant le CSS inline, on a supprimé une requête CSS, mais on a aussi rendu le document HTML plus "lourd" et moins mutualisable par le cache navigateur. Avec un fichier CSS séparé, le navigateur peut le garder en cache pour toutes les pages suivantes ; en inline, chaque page renvoie son propre CSS dans le HTML. Sur un audit EcoIndex (souvent fait en navigation "froide" ou avec des règles de pondération différentes), ce gain de requête peut être inférieur à la perte liée au poids HTML / structure, donc la note peut baisser.

Cependant, en changeant la logique CSS, on augmente fortement le Speed Index

Avant optimisation (page quizzes)

Speed Index
3,7 s

Après optimisation

Speed Index
0,8 s

Cette amélioration est cohérente : en intégrant le CSS directement dans le HTML, le navigateur peut appliquer les styles immédiatement sans attendre une requête supplémentaire vers style.css. On supprime donc une ressource bloquante sur le chemin critique, ce qui accélère l'affichage visuel des premiers éléments et fait chuter fortement le Speed Index (de 3.7 à 0.8). En bref, l'utilisateur voit la page "se dessiner" beaucoup plus vite, même si d'autres métriques globales peuvent évoluer différemment.

<u>Autre optimisation possible :</u>

Lors de l'analyse Website Carbon Calculator, on remarque que le site utilise de l'énergie sans technologies pour réduire l'empreinte carbone.

Le problème d'énergie étant lié au site d'hébergement, il nous est compliqué de pouvoir modifier quoi que ce soit sur l'énergie utilisée et cela, encore plus sur une version d'essai gratuite.

Réflexion sur l'optimisation :

L'une des raisons principales pour lesquelles nous n'avons pas eu besoin de faire une optimisation lourde du code est que l'application avait déjà été conçue avec une logique de sobriété dès le départ. Nous avons volontairement choisi une architecture simple (Flask + SQLite + templates HTML/CSS/JS natifs), sans framework front-end complexe ni dépendances inutiles. Ce choix réduit naturellement le volume de code exécuté, le nombre de requêtes réseau, et la charge de calcul côté client comme côté serveur.

Le code a aussi été pensé pour éviter les surcoûts fonctionnels : pas de traitements asynchrones superflus, pas de bibliothèques "lourdes" juste pour du confort de développement, et une base de données locale adaptée au besoin réel du projet. En d'autres termes, au lieu de corriger a posteriori une application surdimensionnée, nous avons évité en amont les causes classiques de surconsommation numérique. Cela explique pourquoi les optimisations restantes ont surtout été des ajustements ciblés, et non une refonte complète.

Les améliorations réalisées (par exemple sur le chargement des styles) s'inscrivent donc dans une logique fine d'optimisation du chemin critique, pas dans une logique de "sauvetage" des performances. Elles permettent de gagner sur l'expérience utilisateur (Speed Index, affichage initial) tout en gardant une base technique légère. Ce type d'optimisation incrémentale est cohérent avec une démarche Green IT : faire peu, mais faire juste, et mesurer les effets avant d'ajouter de la complexité.

Il est aussi important de souligner qu'optimiser ne signifie pas uniquement "faire baisser une métrique unique". Certaines décisions peuvent améliorer fortement la perception de vitesse (ex. Lighthouse) tout en ayant un impact neutre ou parfois négatif sur d'autres indicateurs plus structurels (ex. EcoIndex). C'est normal : ces outils n'évaluent pas exactement la même chose. Notre approche reste donc pragmatique : conserver une base sobre, arbitrer selon le contexte d'usage, et privilégier les optimisations qui apportent un vrai bénéfice global.

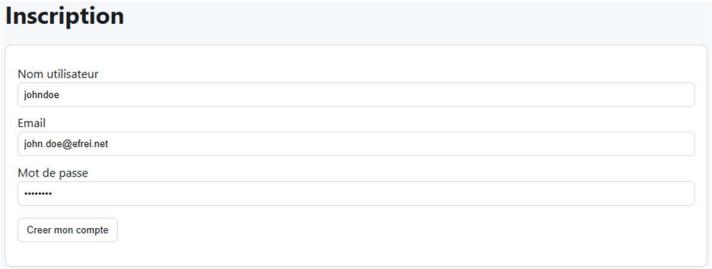
Enfin, cette démarche montre qu'un projet peut être performant et plus responsable sans empiler des technologies complexes. Le résultat vient surtout des choix de conception initiaux : simplicité de la stack, limitation des dépendances, sobriété des fonctionnalités et contrôle du périmètre. C'est précisément cette cohérence de conception qui a limité le besoin d'optimisations tardives et coûteuses.

Section 6. Tests et validation

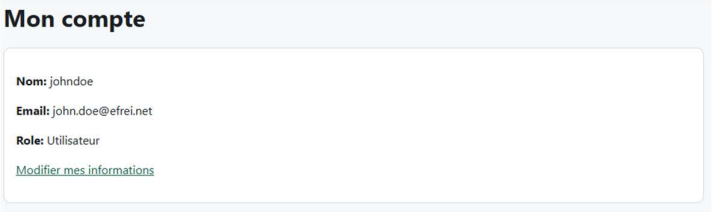
Tableau des scénarios de tests fonctionnels, résultats Lighthouse, captures d'écran.

Tests fonctionnels :

1) Créer un utilisateur valide :

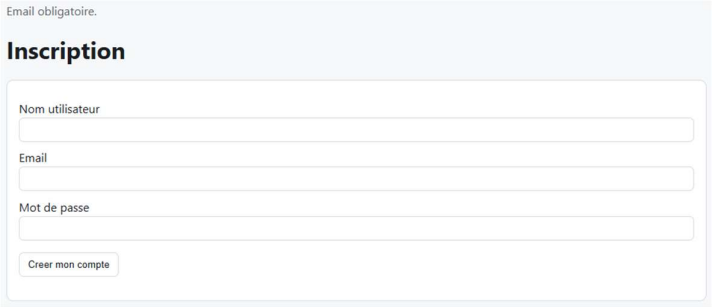


Rédirect vers la page account



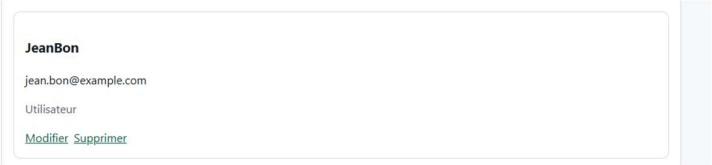
2) Créer un utilisateur (email vide)

Impossible d'aller plus loin : affichage d'un message d'erreur (Email obligatoire)

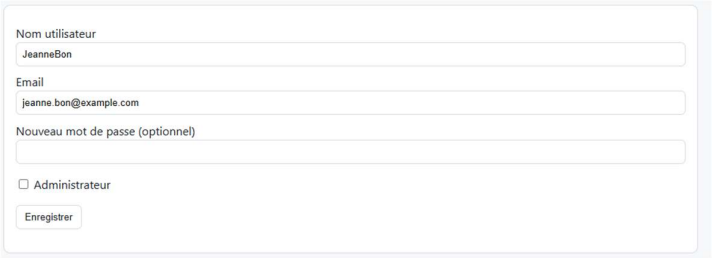


3) Modification compte

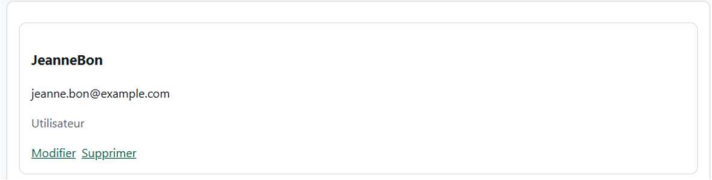
Exemple depuis un compte administrateur (l'utilisateur peut aussi changer son propre compte)



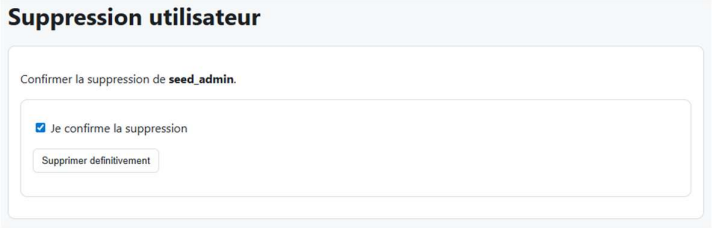
On clique sur modifier



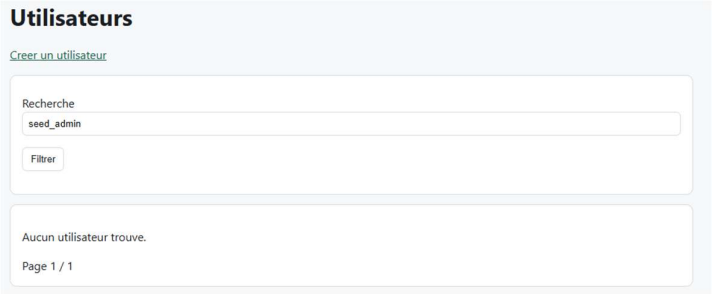
Après avoir modifié ses informations et avoir cliqué sur Engistrer, les informations changent sur la base de données



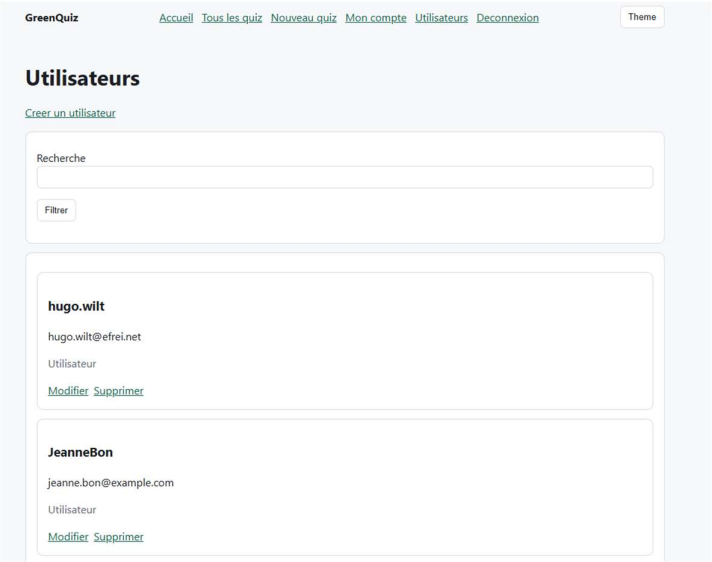
4) Supprimer un utilisateur (Utilisateur supprimé après confirmation)



L'utilisateur est supprimé de la base de données

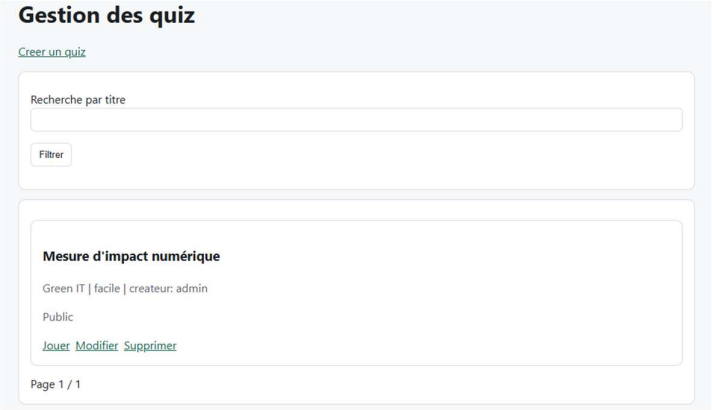


5) Lister des utilisateurs

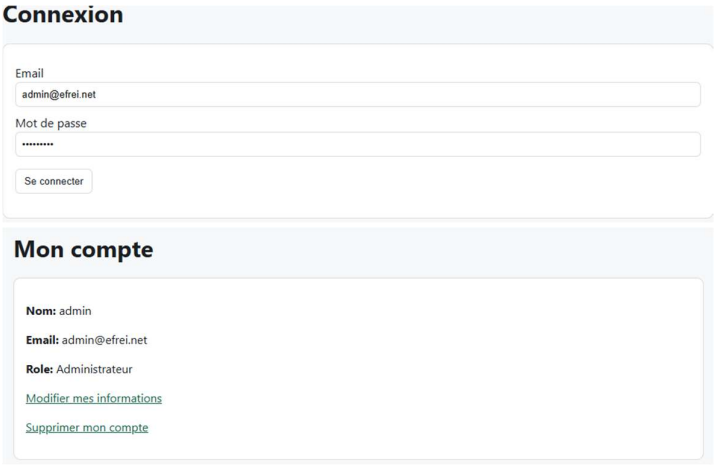


Affichage liste utilisateurs (depuis un compte admin) et possibilité de recherche un utilisateur

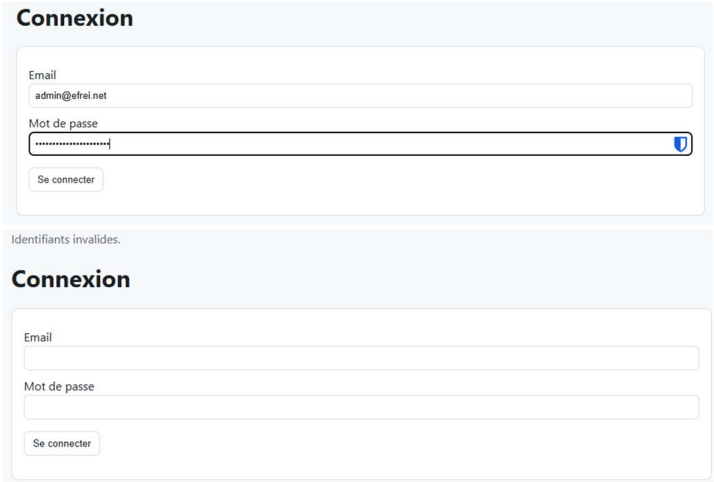
6) Créer une entité métier



7) Connexion avec des identifiants valides



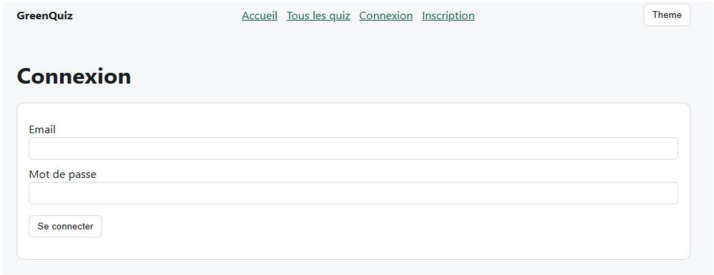
8) Connexion avec mauvais MDP



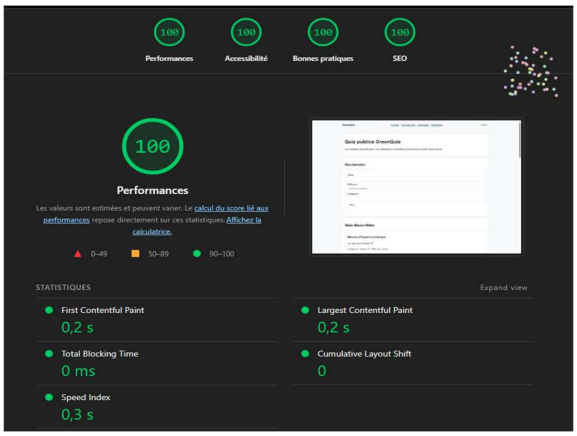
9) Accès page protégée sans connexion

Tentative d'accès page admin en mode invité :

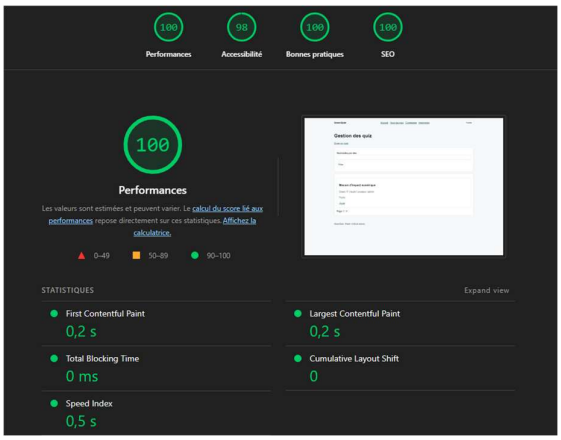
Redirect vers page de connexion



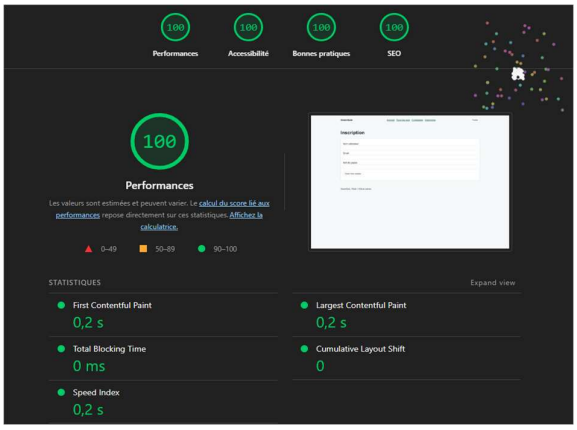
Page d’accueil



Page de quizz

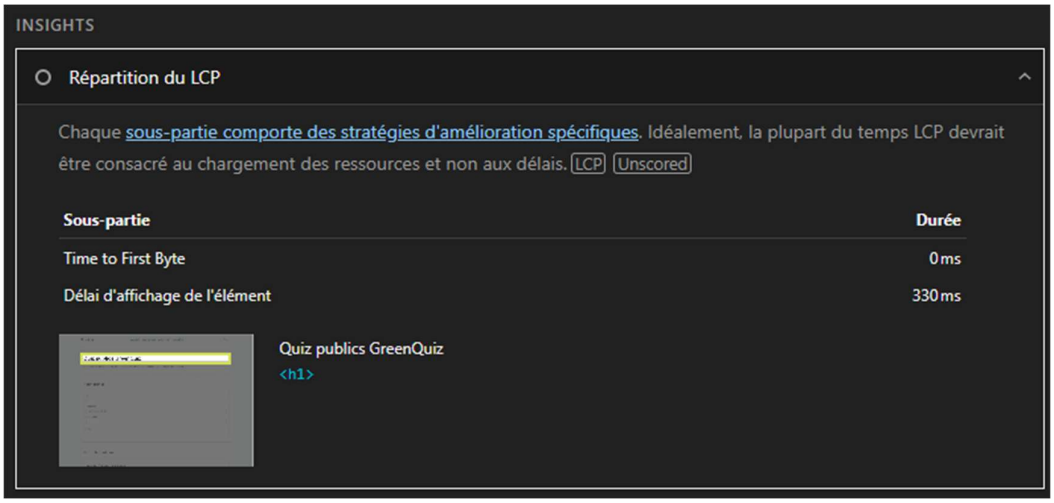


Page d’inscription

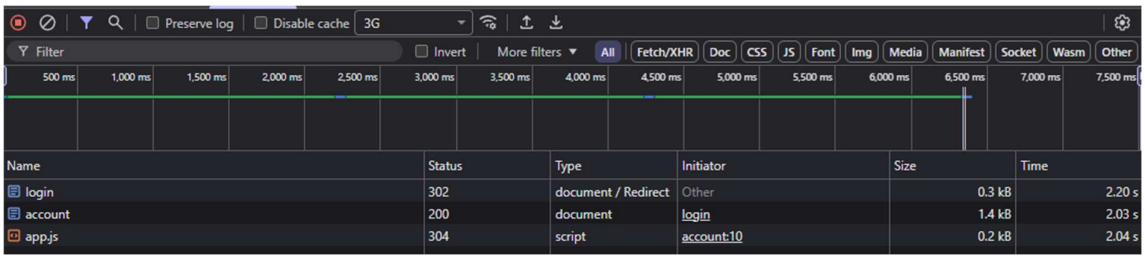


Analyse Insights :

Après la première correction du code (cf : optimisation Speed Index), il n’y a plus de messages d’erreurs sur la partie Insights. Cependant, sur les différentes pages, il reste encore la mention de certaines optimisations secondaires possibles.



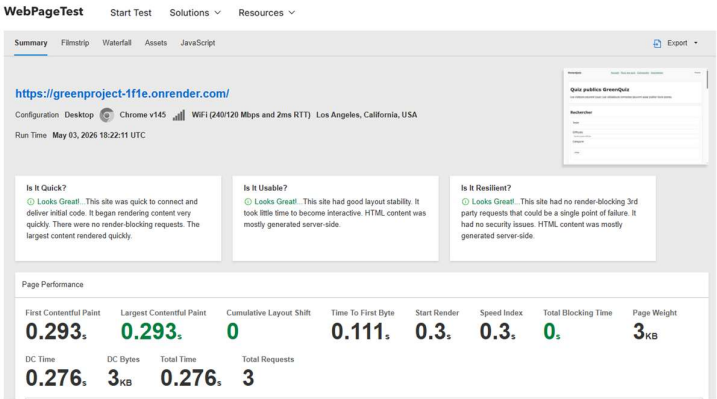
Test de connexion en mode lent simulé :



Malgré un mode lent simulé (3G) grâce aux Dev Tools, le site continue de fonctionner, même en parcourant différentes pages, en se connectant etc.

Malgré un temps accru, le site reste très simple et donc rapide à charger, même avec un réseau très lent.

Analyse WebPageTest :



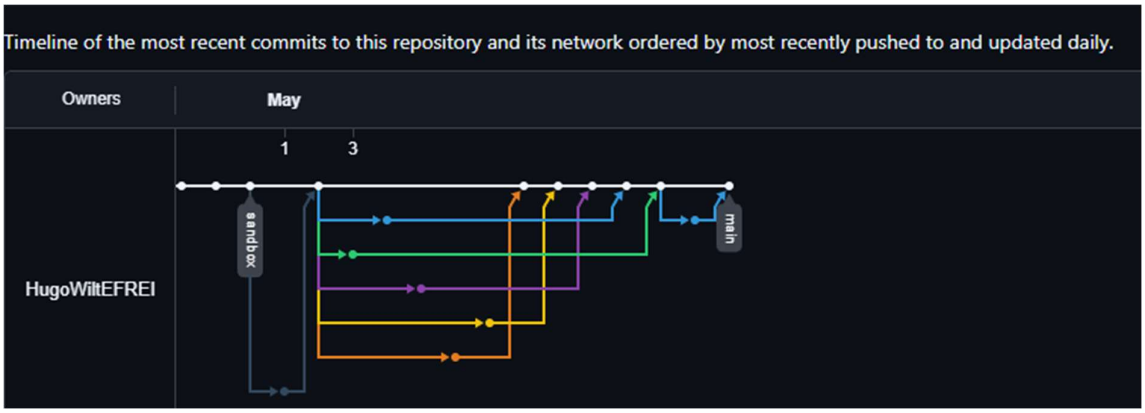
Le chargement est très court et propre : seulement 3 requêtes (document, un fichier static/asset..., et favicon.ico) pour un poids total d'environ 3 KB. Les timings confirment ça : TTFB ~0.11s, Start Render ~0.3s, FCP/LCP ~0.293s, Speed Index ~0.3s, TBT = 0.

Côté ressources bloquantes, il n'y a pas de blocage significatif visible. Le waterfall montre surtout que la première contrainte est la requête HTML initiale (normal), puis le fichier statique démarre rapidement et ne décale pas fortement le rendu. Le badge WebPageTest indique d'ailleurs "no render-blocking requests". Donc, dans cet état, aucune ressource tierce ou lourde ne bloque réellement l'affichage.

Section 7. Organisation de l'équipe et collaboration

Répartition des tâches :

- Hugo W - Coordination projet, développement full-stack, rédaction
- Charles Y - Développement full-stack, rédaction
- Ornella T - Rédaction, Organisation



Section 8. Discussion et conclusion

1. Défis rencontrés

Problème de base de données MySQL. L'erreur initiale (Access denied for user 'root'@'localhost') montre une dépendance à une configuration locale non maîtrisée. MySQL nécessite un service externe actif, un mot de passe configuré, et une base créée au préalable — trop de prérequis pour un mini-projet sobre.

Instabilité de la stack technique Le projet a changé trois fois de stack en cours de route : PHP/MySQL → HTML statique → PHP/SQLite → Flask/SQLite. Chaque migration a introduit de nouveaux bugs (hash scrypt incompatible avec la version Python locale, Internal Server Error à l'inscription), ce qui a allongé inutilement le temps de développement.

Compatibilité Python hashlib.scrypt n'étant pas disponible sur l'environnement local, generate_password_hash() crashait silencieusement. Ce type de bug dépend de la version Python installée et est difficile à anticiper sans environnement virtuel (venv).

2. Compromis entre fonctionnalités et sobriété

Ce qui a été sacrifié au nom de la sobriété

- Suppression de MySQL au profit de SQLite : SQLite ne nécessite aucun serveur, consomme moins de ressources.
- Passage d'une stack PHP complète (CRUD, auth lourde, multiples fichiers) à un backend minimaliste Flask : réduit le nombre de fichiers, la complexité et les requêtes serveur.
- Élimination des pages non essentielles : cohérent avec une démarche Green IT.

Ce qui a été récupéré après avoir été trop sacrifié (tension réelle) Le passage en HTML statique pur a supprimé la base de données et l'authentification — deux fonctionnalités centrales du projet. Il a fallu les réintroduire, ce qui montre qu'un site Green IT sobre ne signifie pas un site sans fonctionnalités. La sobriété doit porter sur l'*implémentation*, pas sur l'*absence de features*.

Tension non résolue Flask + SQLite reste une solution avec un interpréteur Python qui tourne en permanence. Un site PHP natif + SQLite aurait pu être encore plus léger (exécution à la demande, pas de processus permanent). Le choix Flask a été guidé par la préférence utilisateur, pas strictement par la sobriété.

3. Pistes d'amélioration

Technique

- Utiliser un venv Python dès le départ pour isoler les dépendances et éviter les bugs de compatibilité comme le problème scrypt.
- Ajouter un fichier runtime.txt (python-3.11.x) pour figer la version Python et garantir la reproductibilité en local et en production.
- Pour le déploiement, préférer Railway avec volume persistant à Render en SQLite, pour ne pas perdre les données à chaque redémarrage.
- Minifier les fichiers CSS/JS pour réduire le poids des assets servis.

Green IT

- Mesurer la consommation réelle du serveur Flask avec powermetrics ou codecarbon pour quantifier le gain par rapport à la version PHP/MySQL initiale — ce serait un argument fort dans le rapport.
- Envisager un cache HTTP côté serveur (headers Cache-Control) pour éviter de regénérer les pages statiques à chaque requête.
- Documenter le nombre de requêtes SQL par page pour montrer une démarche d'optimisation concrète.

Section 9. Annexes

Backlog final (GreenQuiz)

Terminé

- Authentification : inscription, connexion, déconnexion
- Gestion du compte : consultation, modification, suppression de son compte
- Gestion utilisateurs (admin) : lister, créer, modifier, supprimer
- Règle admin forcée pour admin@efrei.net
- Validation serveur des formulaires (dont email vide -> message explicite)
- Gestion des quiz : créer, modifier, supprimer, lister
- Passage de quiz + calcul du score
- Historique des sessions de quiz
- Base SQLite structurée + index
- Seed de plusieurs quiz Green IT
- Correctif déploiement Render (0.0.0.0 + PORT)
- Optimisation chemin critique CSS (réduction blocage rendu)

Améliorations souhaitables (post-MVP)

- Pagination/filtres plus avancés sur quiz et utilisateurs
- Tests automatisés persistants (suite pytest)
- Politique de sauvegarde/restauration SQLite
- Migration vers serveur WSGI (gunicorn) pour prod
- Tableau de bord admin (stats d'usage, scores, activité)
- Réinitialisation de mot de passe / email de récupération

Hors périmètre actuel

- OAuth / SSO
- Multi-langue
- API publique complète
- Recommandation intelligente de quiz